Running Job on RCC systems using the SLURM Scheduler

Debasmita Samaddar

Nov 4, 2025

Agenda

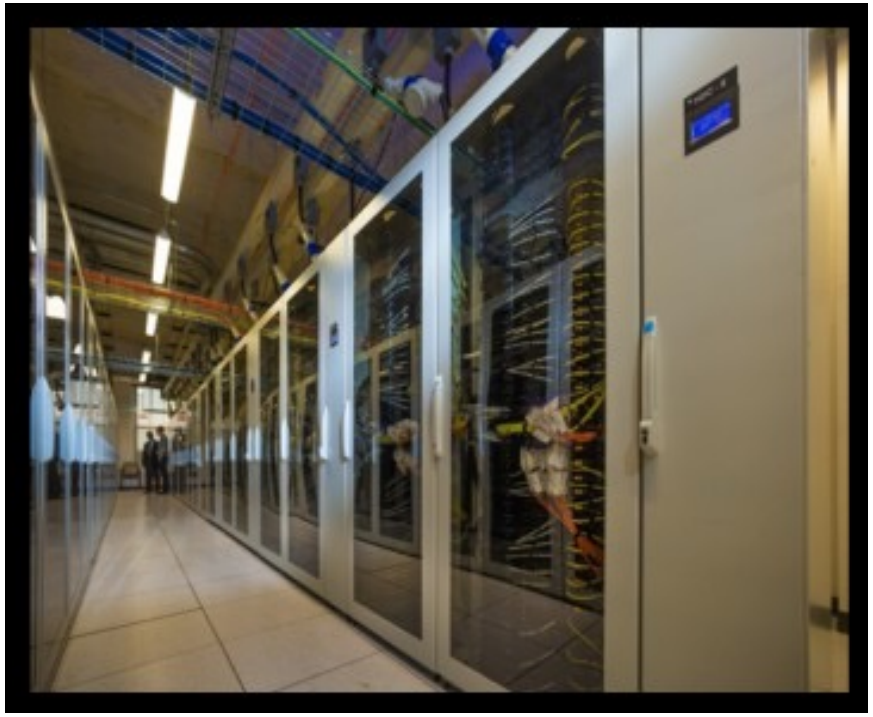
- The RCC Midway compute systems
- Using Slurm (Simple Linux Utility for Resource Management) to submit jobs to the RCC Midway systems

=> Goals of this workshop

  - Learn the basics of Slurm
  - Be comfortable submitting jobs on a Midway HPC systems
  - Understand queue priority
  - Be comfortable checking if jobs succeeded/failed
  - Know how to fix common errors

Understanding the RCC Compute Ecosystem

Midway2 and 3 are a collection of many compute systems and storage with various architectures coupled together in one system.



Slurm is the software used to manage the workload on Midway 2 & 3.

Some definitions

- A **processor** is a small chip that responds to and processes the basic instructions that drive a computer. The term *processor* is used interchangeably with the term **central processing unit** (CPU)
- **Core**: The smallest compute unit that can run a program
- **Socket:** A compute unit, packaged as one and usually made of a single chip often called processor. Modern sockets carry many cores (10, 14, or 20, 24, 28, etc. on most servers)
- **Node:** A stand-alone computer system that contains one or more sockets, memory, storage, etc. connected to other nodes via a fast network interconnect.

The RCC compute cluster

Computing hardware

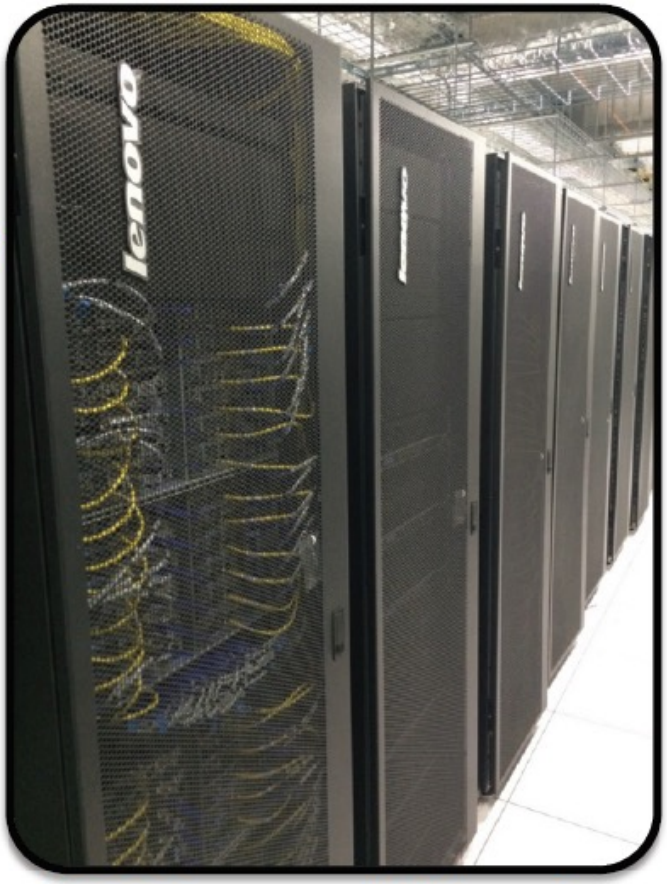
Midway2

367 nodes total

- **342** tightly coupled Broadwell nodes (10,360 cores)
  Two intel E5-2680v4 processors per node (14 core/proc)
  155 nodes have EDR network card
  187 nodes have FDR network card
- **6** NVidia Tesla K80 GPU nodes (4 GPU cards/node)
- **5** large shared memory nodes (512GB each)
- **14** dual socket loosely-coupled Broadwell nodes

Cluster Partnership Program: 1000+ nodes

- **900+** tightly coupled infiniband nodes
- **20+** Big memory nodes
- **100+** Nvidia GPU nodes



RCC Manages 1515 nodes (35,904 cores)

Midway3 at RCC, launched in March 2021

Compute Nodes

There are a total of 210 standard Intel Cascade Lake CPU-only nodes available to all users.

Each node has the following base components:

- 2x Intel Xeon Gold 6248R (48 cores per node)
- 192 GB of conventional memory
- HDR InfiniBand (100 Gbps) network card

Storage

Distributed file system storage: 2.2 PB of high performance GPFS storage

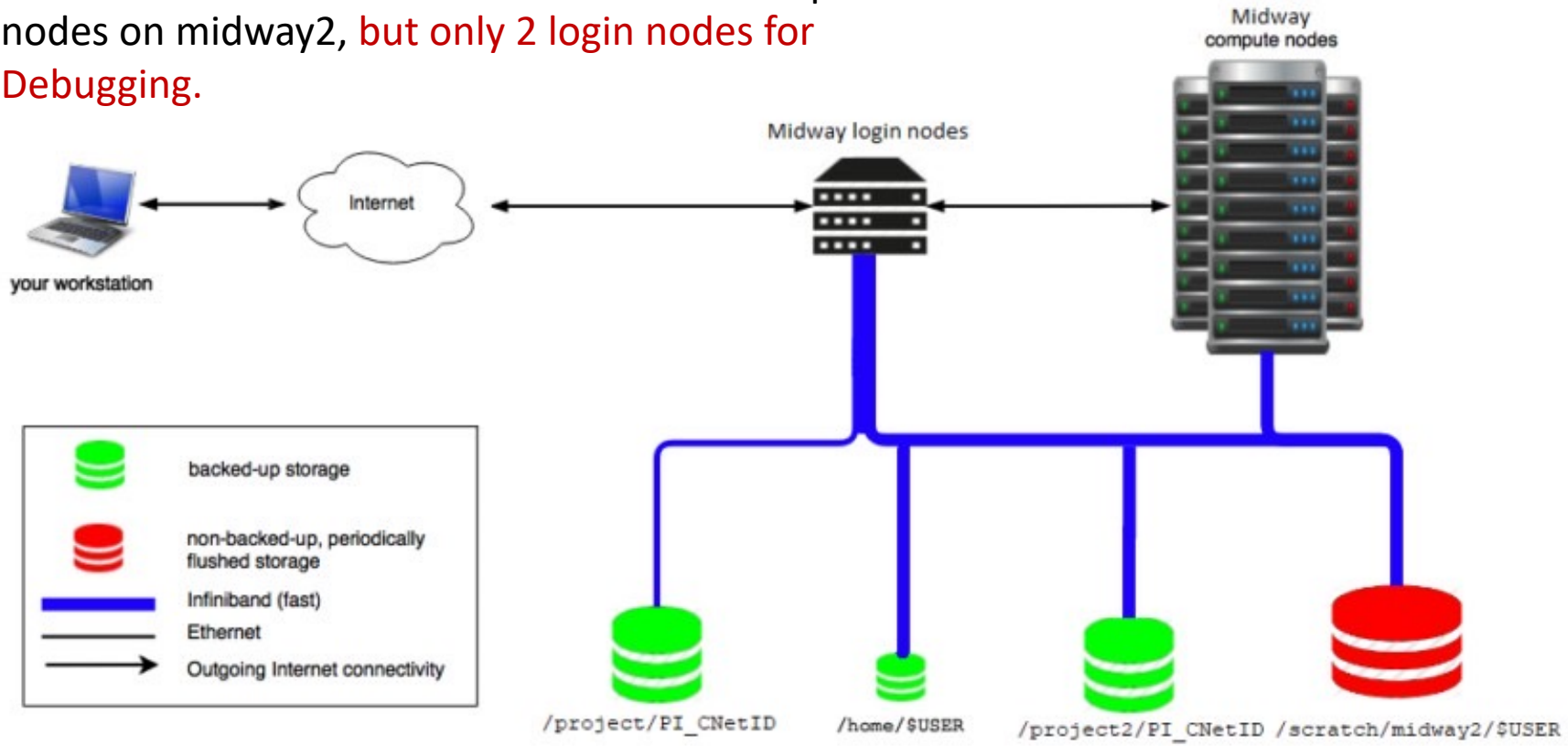
CPU Microarchitecture:	**Intel Xeon 6248R**
Number of Cores per Node:	**48**
Memory per Node:	**192 GB**
Local SSD storage per node:	**960 GB**
Network Bandwidth:	**100 Gbps**

CPU Specifications

# of Cores	24
# of Threads	48
Processor Base Frequency	3.00 GHz
Max Turbo Frequency	4.00 GHz
Cache	35.75 MB
# of UPI Links	2
TDP	205 W

Schematic of the Midway Cluster

There are about more than 1300+ nodes compute nodes on midway2, but only 2 login nodes for Debugging.



RCC Midway2 Storage

High-Capacity storage: /project2

- **3.8 PB** of storage
- Backed up to tape system
- **7 daily and 4 weekly snapshots** located at /snapshots/project2
- 7 days grace period on over quota

High Performance storage: /scratch/midway2

- **190 TB** usable
- Not backed up
- 100 GB user soft quota
- 30 days grace period on over quota

Home directory space: /home

- **61 TB** of capacity
- Each user has 30 GB quota
- 7 days grace period on over quota
- **7 daily and 2 weekly snapshots** located at /snapshots/home



RCC Midway3 Storage

High-Capacity storage: /project

- **3.7 PB** of storage
- **7 daily and 4 weekly snapshots** located at /snapshots/project
- 7 days grace period on over quota

High Performance storage: /scratch/midway3

- **355 TB** usable
- Not backed up
- 100 GB user soft quota

Home directory space: /home

- Each user has 30 GB quota
- 7 days grace period on over quota
- **7 daily and 4 weekly snapshots** located at /snapshots/home

How to Run jobs at RCC

Either Interactively or submitting jobs to a queue using Slurm

Please clone the repo:

git clone https://github.com/rcc-uchicago/Slurm_workshop_MW3.git

A Key point to remember

There are about more than 1300+ nodes compute nodes on midway3, but only 2 login nodes.

This means you are sharing the login nodes with many other users at once. Running intensive programs on the login nodes causes the login nodes to be slow for all other users.

- login nodes are for editing files, compiling, moving files, changing permissions, and other non-intensive tasks.
- We recommend to use *sinteractive* for interactive runs
- For long running jobs => submit them to the queue

Let’s practice the Exercise (Ex-1 in Repo)

cd Slurm_workshop_MW3/Ex-1
sbatch jobid.sbatch

Slurm: Some key terms to remember

A **job** is the resources you are using and the code you are running

The **queue** in Slurm is all RUNNING and all PENDING jobs
To see every job in the queue on Midway2, use the command

```
squeue
```

To see your jobs in the queue

```
squeue -u <cnetid>
```

or

```
myq
```

What goes into a batch script?

A batch script is list of instructions for slurm.

```
#!/bin/bash

# Here is a comment
#SBATCH --time=1:00:00

#SBATCH -nodes=1
#SBATCH -ntasks-per-node=1
#SBATCH --mem-per-cpu=2000
#SBATCH -job-name=MyJob
#SBATCH -output= MyJob-%j.out
#SBATCH -error=MyJob-%j.err

module load <module name>
#Run your code
```

What goes into a batch script?

A batch script is list of instructions for slurm.

```
#!/bin/bash

# Here is a comment
#SBATCH --time=1:00:00

#SBATCH -nodes=1
#SBATCH -ntasks-per-node=1
#SBATCH --mem-per-cpu=2000
#SBATCH -job-name=MyJob
#SBATCH -output= MyJob-%j.out
#SBATCH -error=MyJob-%j.err

module load <module name>
#Run your code
```

This #! is a shebang

It tells operating system to use /bin/bash with this script

What goes into a batch script?

A batch script is list of instructions for slurm.

```
#!/bin/bash

# Here is a comment
#SBATCH --time=1:00:00

#SBATCH -nodes=1
#SBATCH -ntasks-per-node=1
#SBATCH --mem-per-cpu=2000
#SBATCH -job-name=MyJob
#SBATCH -output= MyJob-%j.out
#SBATCH -error=MyJob-%j.err

module load <module name>
#Run your code
```

is a comment
everything after # is ignored by bash

What goes into a batch script?

A batch script is list of instructions for slurm.

```
#!/bin/bash

# Here is a comment
#SBATCH --time=1:00:00

#SBATCH -nodes=1
#SBATCH -ntasks-per-node=1
#SBATCH --mem-per-cpu=2000
#SBATCH -job-name=MyJob
#SBATCH -output= MyJob-%j.out
#SBATCH -error=MyJob-%j.err

module load <module name>
#Run your code
```

#SBATCH is a directive

It is a comment in Bash

#SBATCH is only relevant to slurm:
sbatch my_script.sh

What goes into a batch script?

A batch script is list of instructions for slurm.

```
#!/bin/bash

# Here is a comment
#SBATCH --time=1:00:00

#SBATCH -nodes=1
#SBATCH -ntasks-per-node=1
#SBATCH --mem-per-cpu=2000
#SBATCH -job-name=MyJob
#SBATCH -output= MyJob-%j.out
#SBATCH -error=MyJob-%j.err

module load <module name>
#Run your code
```

#SBATCH is a directive

It is a comment in Bash

#SBATCH is only relevant to slurm:
sbatch my_script.sh

To comment out directives, break the pattern, e.g.
##SBATCH
SBATCH

What goes into a batch script?

A batch script is list of instructions for slurm.

```
#!/bin/bash

# Here is a comment
#SBATCH --time=1:00:00

#SBATCH -nodes=1
#SBATCH -ntasks-per-node=1
#SBATCH --mem-per-cpu=2000
#SBATCH -job-name=MyJob
#SBATCH -output= MyJob-%j.out
#SBATCH -error=MyJob-%j.err

module load <module name>
#Run your code
```

Instructions for Slurm must go at the top of the script

Any #SBATCH lines you put after your program will be ignored

What goes into a batch script?

A batch script is list of instructions for slurm.

```
#!/bin/bash

# Here is a comment
#SBATCH --time=1:00:00

#SBATCH -nodes=1
#SBATCH -ntasks-per-node=1
#SBATCH --mem-per-cpu=2000
#SBATCH -job-name=MyJob
#SBATCH -output= MyJob-%j.out
#SBATCH -error=MyJob-%j.err

module load <module name>
#Run your code
```

Slurm has some variables you can use. %j is the job number. When the job runs %j will be expanded to the job number. In this example %j is used in the output file and error file names:

MyJob-13571056.out
MyJob-13571056.err

%j is unique. By using %j in your filenames you guarantee a unique file name, which means you won't accidentally overwrite previous output.

What goes into a batch script?

A batch script is list of instructions for slurm.

```
#!/bin/bash

# Here is a comment
#SBATCH --time=1:00:00          => Time your job is allowed to run

#SBATCH -nodes=1                => Number of nodes to run on
#SBATCH -ntasks-per-node=1      => Number of cores on each node to use
#SBATCH --mem-per-cpu=2000      => Memory per cpu => 2000Mb or 2Gb
#SBATCH -job-name=MyJob         => Name of the job.
#SBATCH -output= MyJob-%j.ou    => Job output file behaves as stdout for the code.
#SBATCH -error=MyJob-%j.err     => Error file. behaves as stderr for the code.

module load <module name>       => Load any modules you need for your application
#Run your code                  => run the code you want
```

Running batch jobs using a Submission Script

- A simple job submission script (saved as python.sbatch):

Slurm

```
#!/bin/bash
#SBATCH --job-name=first_python_job
#SBATCH --output=first_python_job_%j.out
#SBATCH --error=first_python_job_%j.err
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --mem-per-cpu=2000M
#SBATCH --partition=broadwl
#SBATCH --reservation=kicpworkshop-cpu
```

Your Job

```
#SBATCH --time=00:30:00
module load python
python hello_world.py
echo "job finished at `date`"
```

- • To submit the above script:
  - sbatch python.sbatch

Summary of partitions on Midway3

sinfo -s

```
[dsamaddar@midway3-login4 Ex-1]$ sinfo -s
PARTITION          AVAIL  TIMELIMIT   NODES(A/I/O/T) NODELIST
test                  up   infinite   393/159/50/602 beagle3-[0001-0044],beagle3-bigmem[1-4],climate-[001-048],midway3-[0001-0216,0219
-0227,0231-0308,0310-0324,0329-0427,0429-0442,0444-0456,0501-0562]
caslake*              up   infinite      198/0/0/198 midway3-[0001-0002,0008-0019,0022-0038,0047-0156,0159-0199,0201-0216]
gpu                   up   infinite         8/3/0/11 midway3-[0277-0286,0294]
build                 up   infinite          1/0/0/1 midway3-0200
bigmem                up   infinite          1/1/0/2 midway3-[0317-0318]
amd                   up   infinite       30/10/0/40 midway3-[0501-0539,0549]
amd-hm                up   infinite          0/1/0/1 midway3-0541
beagle3               up   infinite        35/8/1/44 beagle3-[0001-0044]
beagle3-bigmem        up   infinite          4/0/0/4 beagle3-bigmem[1-4]
kicp                  up   infinite          2/4/0/6 midway3-[0222-0227]
kicp-gpu              up   infinite          0/1/0/1 midway3-0297
ssd                   up   infinite        3/15/0/18 midway3-[0259-0276]
ssd-gpu               up   infinite          0/1/0/1 midway3-0315
lgagliardi-ld         up   infinite         4/6/0/10 midway3-[0157-0158,0332,0343-0345,0438-0441]
lgagliardi-gpu        up   infinite          1/0/0/1 midway3-0293
lgagliardi-hm         up   infinite          3/0/0/3 midway3-[0308,0450,0557]
lgagliardi-amd        up   infinite         8/3/0/11 midway3-[0542-0545,0552-0556,0561-0562]
lgagliardi-net        up   infinite          1/0/0/1 midway3-0540
ent                   up   infinite          0/2/0/2 midway3-[0452-0453]
vitelli               up   infinite          1/0/0/1 midway3-0219
vitelli-amd           up   infinite          2/0/0/2 midway3-[0546-0547]
vitelli-gpu           up   infinite          0/1/0/1 midway3-0307
andrewferguson        up   infinite          0/1/0/1 midway3-0220
odube                 up   infinite          0/1/0/1 midway3-0221
cobey                 up   infinite          0/5/0/5 midway3-[0231-0235]
cobey-hm              up   infinite          0/6/0/6 midway3-[0324,0329-0331,0375,0429]
damazz-hm             up   infinite          0/1/0/1 midway3-0376
gagalli-csl2          up   infinite       19/11/0/30 midway3-[0236-0249,0400-0415]
gagalli-gpu           up   infinite          1/2/0/3 midway3-[0298,0377-0378]
dinner                up   infinite          1/7/0/8 midway3-[0250-0257]
dinner-hm             up   infinite          0/1/0/1 midway3-0288
depablo               up   infinite        0/10/0/10 midway3-[0333-0342]
depablo-gpu           up   infinite          4/3/1/8 midway3-[0295,0299-0304,0372]
arzhetsky             up   infinite          0/1/0/1 midway3-0258
neiman                up   infinite          0/1/0/1 midway3-0287
```

NODES(A/I/O/T) : Nodes (Allocated/Idle/Other/Total)

e.g: To check the caslake partition: sinfo -s |grep caslake

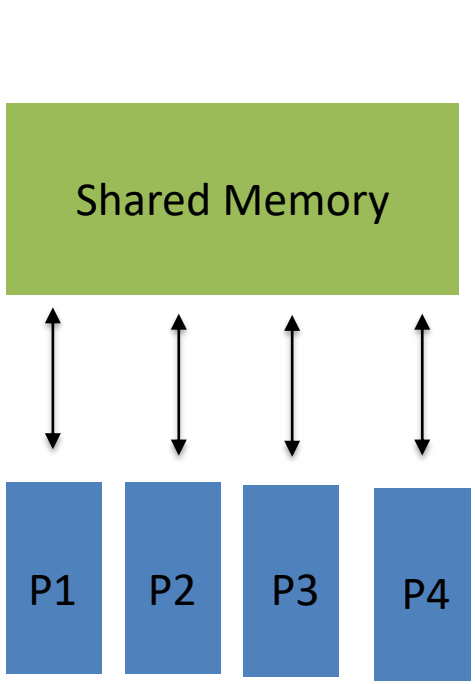
Running Interactive jobs

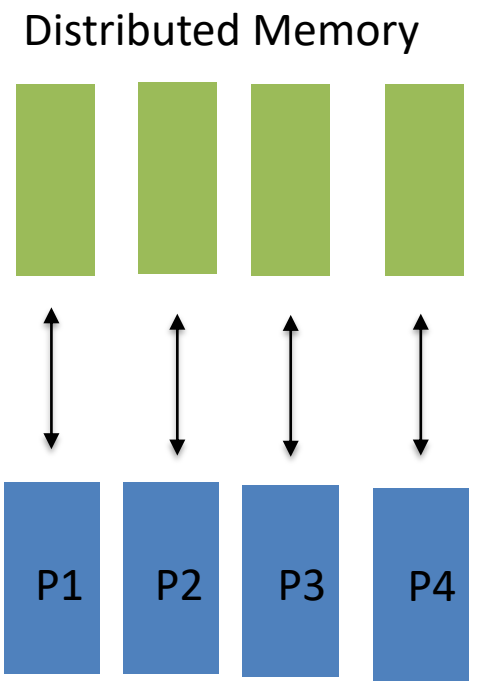
- login directly to a node
  - Login to midway3.rcc.uchicago.edu
  - Run the job at the command prompt

- Run interactively using sinteractive
  - Uses Slurm to provide access to dedicated node(s) to which you can login directly
  - To use sinteractive:

```
sinteractive --time=01:00:00 --nodes=1 --ntasks=2 --mem-per-cpu=1000 --partition=caslake --account=pi-centID
```

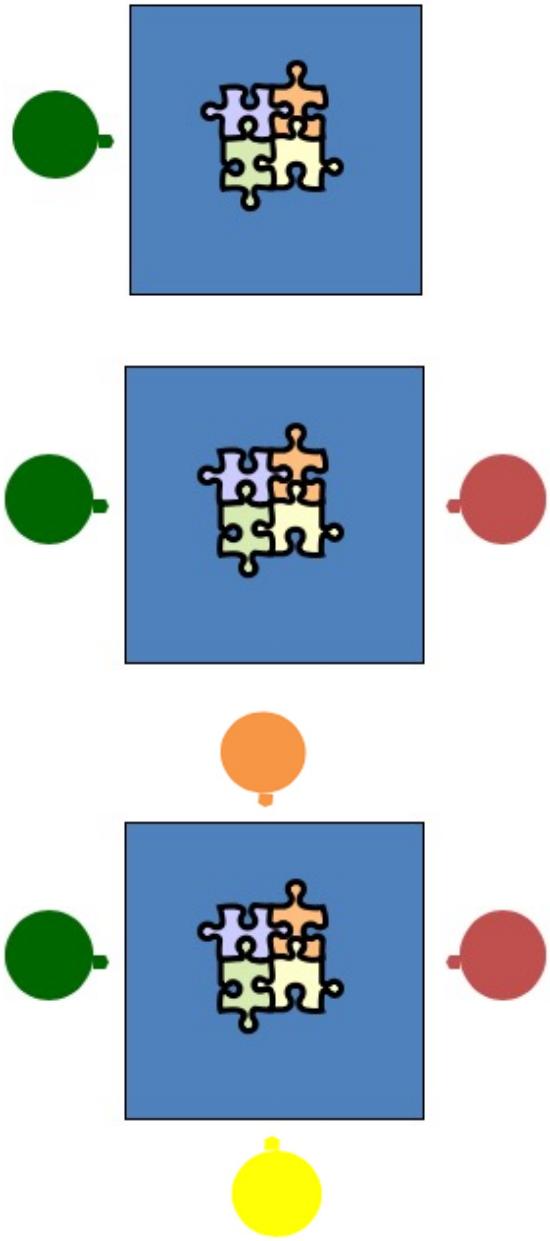
Parallel Computing





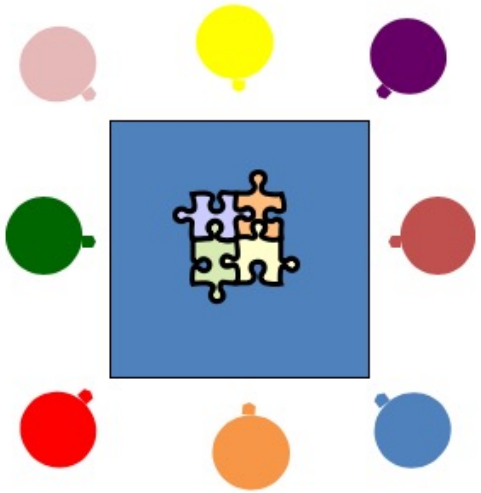
The Jigsaw Puzzle

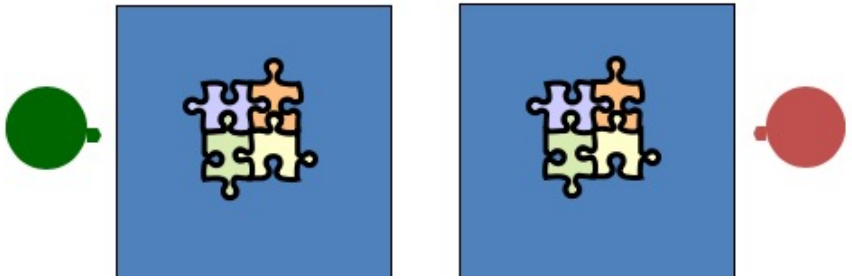
- Serial Computing
  - You sit down at a table by yourself and put together all 1000 pieces, one after the next.
- Shared Memory Parallelism (OpenMP)
  - If your friend sits across from you, then he can work on his half of the puzzle and you can work on yours
  - Once in a while, you'll both reach into the pile for the same piece (you will **contend** for the same resource) which causes a little slowdown
  - Each person works on their quadrant of the puzzle, we should get a 4x speedup, right? But, there will be a lot more contention for pieces and a lot more communication



Parallel The Jigsaw Puzzle

- But...
  - Let's assume the 8 of you are extremely good at working together
  - Maybe you can get an almost 8x speedup
- Distributed Parallelism (MPI – Message Passing Interface)
  - You sit at table 1 and your friend sits at table 2
  - PRO: Plenty of elbow room &You can work without contention for resources
  - CON: Communication is much more difficult. You need to carry pieces from one table to the other to assemble them





Let’s practice the Exercise (Ex-2 in Repo)

How to submit OpenMP jobs?

```
#!/bin/bash

#Here is a comment
#SBATCH --time=1:00:00
#SBATCH -partition=caslake
#SBATCH -nodes=1
#SBATCH -ntasks-per-node=8
#SBATCH --mem-per-cpu=1600
#SBATCH -job-name=MyJob
#SBATCH -output= MyJob-%j.out
#SBATCH -error=MyJob-%j.err
make -f Makefile
module load <module name>
export OMP_NUM_THREADS=8
#Run your code i.e Running the executable
./norm_prog
```

Specify number of cores > 1.

OMP_NUM_THREADS is an environment variable.

sbatch omp_job.sbatch

Please do the Exercise :Ex-3 in Repo for MPI

How to submit Parallel MPI jobs?

```
#!/bin/bash

#Here is a comment
#SBATCH --time=1:00:00
#SBATCH -job-name=MyJob
#SBATCH -output= MyJob-%j.out
#SBATCH -error=MyJob-%j.err
#SBATCH -partition=broadwl
#SBATCH -nodes=2
#SBATCH -ntasks-per-node=4
#SBATCH --mem-per-cpu=2000
module load openmpi
module load <module name>
#Run your code
mpirun ./my_executable
```

Specify number of nodes > 1.

Specify number of cores >= 1.

Load OPENMPI MPI library or IntelMPI:
module load openmpi

Please do the Exercise;
Ex-4 in Repo

How to submit GPU jobs?

```
#!/bin/bash

#Here is a comment
#SBATCH --time=1:00:00
#SBATCH -partition=gpu
#SBATCH -gres=gpu:1
#SBATCH -nodes=1
#SBATCH -ntasks-per-node=8
#SBATCH --mem-per-cpu=2000
#SBATCH -job-name=MyJob
#SBATCH -output= MyJob-%j.out
#SBATCH -error=MyJob-%j.err

module load <module name>
#Run your code
./my_executable
```

Specify partition gpu

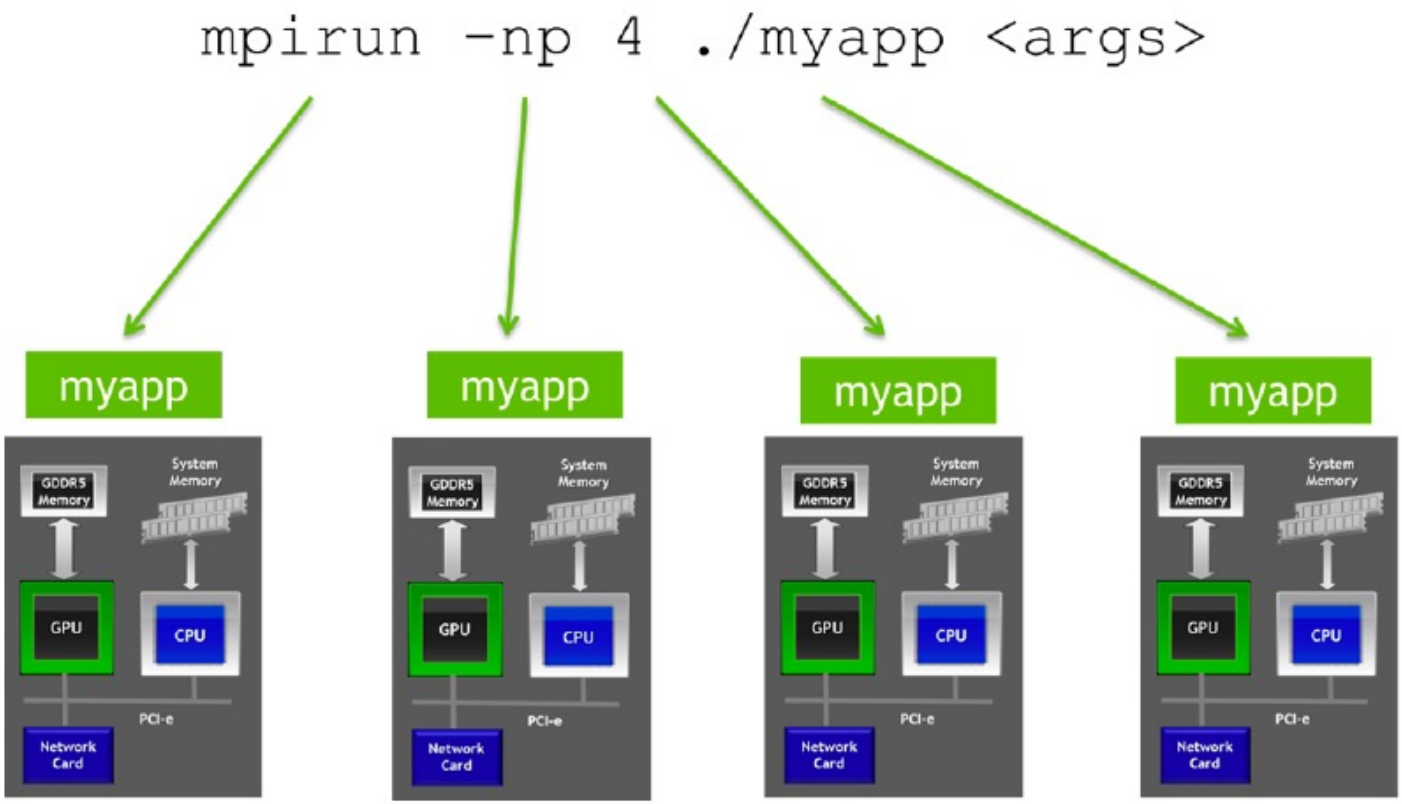
Specify number of gpus, like *gpu*:1

Module load cuda/10.2

Please do the Exercise;
Ex-5 in Repo

How to submit MPI + GPU jobs?

CUDA Aware MPI



How to submit MPI + GPU jobs?

chmod +774 driver.sh
./driver.sh

Compilation

```
#!/bin/bash

module load openmpi/3.1.2
module load cuda/10.1

# Compiling the device code
nvcc -c dev.cu
#Compiling the host code
mpicc -c hostname.c

# Linking the host and device code
mpicc -o HostMap dev.o hostname.o -lcudart

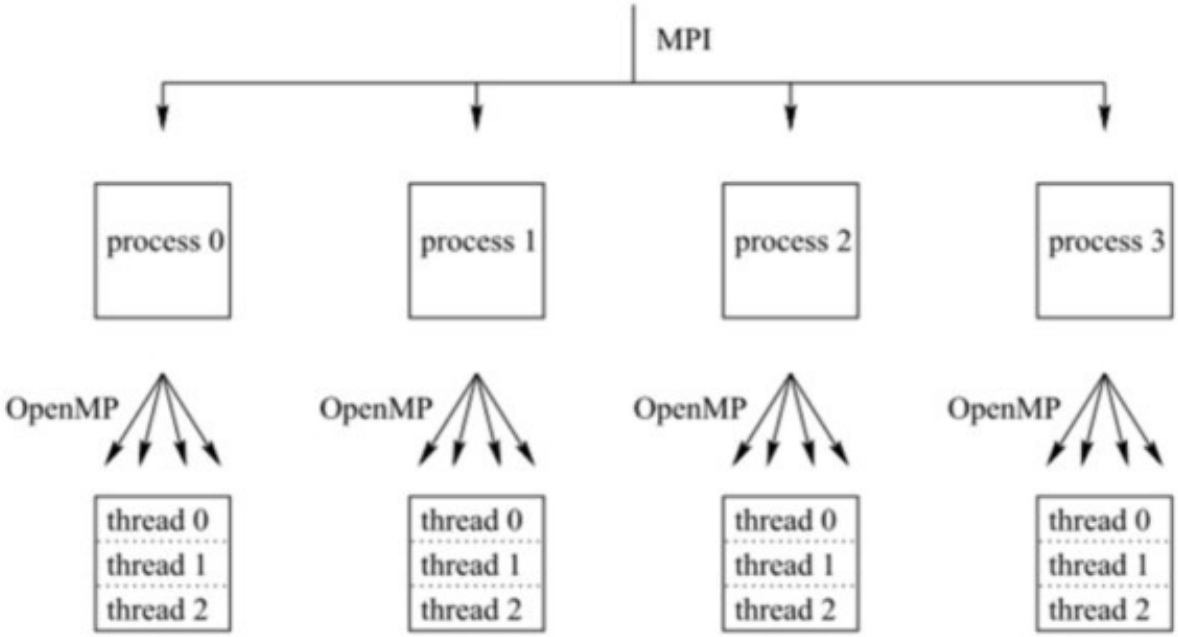
#Submitting the job as batch script
sbatch mpijob.sh
```

Job Submission

```
#!/bin/bash
#SBATCH -t 00:30:00
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=4
#SBATCH --partition=gpu2
#SBATCH --gres=gpu:2
#SBATCH --job-name=MyJob
#SBATCH --output=MyJob-%j.out
#SBATCH --error=MyJob-%j.err
#SBATCH --qos=stafftest
mpirun ./HostMap
```

Please do the Exercise;
Ex-6 in Repo

How to submit MPI + OpenMPI jobs?



```
#!/bin/bash
#SBATCH --job-name=hybrid
#SBATCH --output=hybrid_%j.out
#SBATCH --error=hybrid_%j.err
#SBATCH --time=00:10:00
#SBATCH --ntasks=4
#SBATCH --cpus-per-task=8
#SBATCH --account=rcc-staff
#SBATCH --partition=caslake
##SBATCH --constraint=edr

ulimit -l unlimited
ulimit -u 10000

# Load the default OpenMPI module.
module load openmpi

make -f Makefile

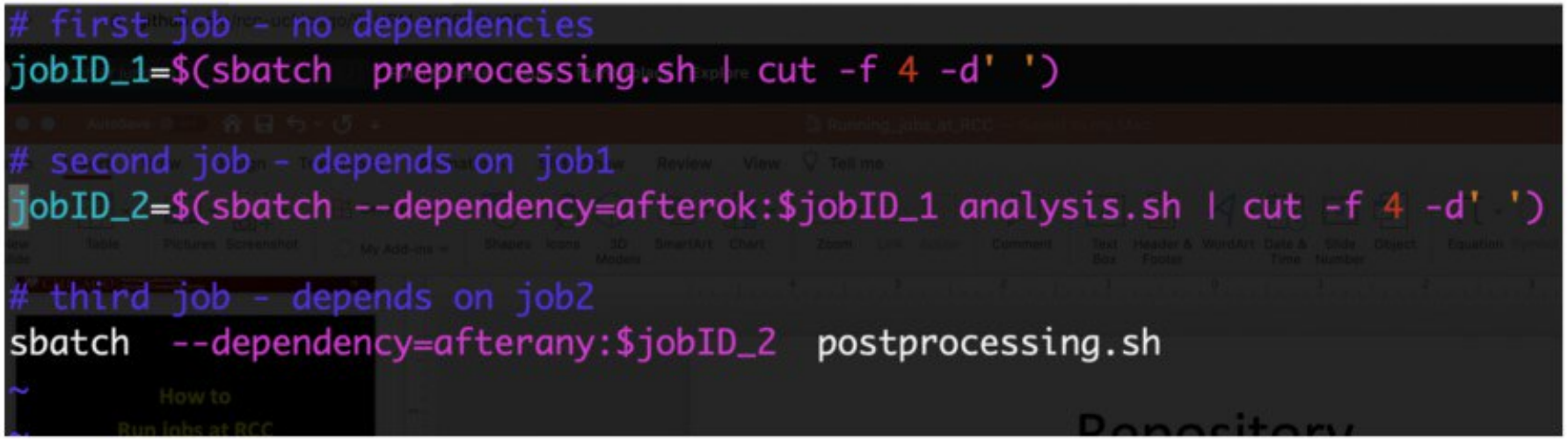
# Set OMP_NUM_THREADS to the number of CPUs per task we asked for.
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
```

sbatch hybrid.sbatch

How to submit dependent jobs

SLURM Rule:

sbatch --dependency=type:job_id jobfile



How to submit dependent jobs

after	This job can begin execution after the specified jobs have begun execution
afterany	This job can begin execution after the specified jobs have terminated.
aftercorr	A task of this job array can begin execution after the corresponding task ID in the specified job has completed successfully
afternotok	This job can begin execution after the specified jobs have terminated in some failed state
afterok	This job can begin execution after the specified jobs have successfully executed
singleton	This job can begin execution after any previously launched jobs sharing the same job name and user have terminated

Please do the Exercise;

Ex-7 in Repo

chmod +774 dependent-jobs.sh
./dependent-jobs.sh

How to submit Parallel batch jobs

```
#!/bin/sh
#SBATCH --time=01:00:00
#SBATCH --partition=caslake
#SBATCH --ntasks=28
#SBATCH --mem-per-cpu=2G # NOTE DO NOT USE THE --mem= OPTION
# Load the default version of GNU parallel. module load parallel
# set the max number of processes (which determine the max per processor)
ulimit -u 10000
# This specifies the options used to run srun. The "-N1 -n1" options are
# used to allocates a single core to each task.
srun="srun --exclusive -N1 -n1”
#Run GNU parallel
parallel="parallel --delay 0.2 -j $SLURM_NTASKS --joblog runtask.log --resume"
# Run a script, runtask.sh, using GNU parallel and srun.
$parallel "$srun ./runtask.sh arg1:{1} > runtask.sh.{1}" ::: {1..128}
# Note that if your program does not take any input, use the -n0 option to call the
parallel command: # # $parallel -n0
"$srun ./run_noinput_task.sh > output.{1}" ::: {1..128}
```

How to submit Parallel batch jobs

Please do the Exercise;
Ex-8 in Repo

chmod +774 runtask.sh
sbatch parallel.sbatch

What resources should I ask for?

This depends on the code you are running

What resources should I ask for?

This depends on the code you are running

Nodes/Cores

- Question: is your code parallel? You will need to find out if your code can
  - Run on multiple cores? Run across multiple nodes?
  - Check if your code is threaded, multiprocessor, MPI
- Question: Is your code serial?
  - This means it can only make use of one core

What resources should I ask for?

This depends on the code you are running

Wall Time

Make an estimate of your job run and add a bit.

e.g. if think your code will take an hour, give it 1 hour and 30 min

- If your job runs out of time, your job will be killed, so
- be accurate with your estimate without going below.

What resources should I ask for?

This depends on the code you are running

Memory

For memory, this can take some trial and error. You can ask for a lot, then measure your usage. If you have asked for more memory and then reduce your memory with the next job.

- To ask for all the memory available on a node, use #SBATCH --mem=0

What if I need an entire node or specific features?

Add this in your batch script

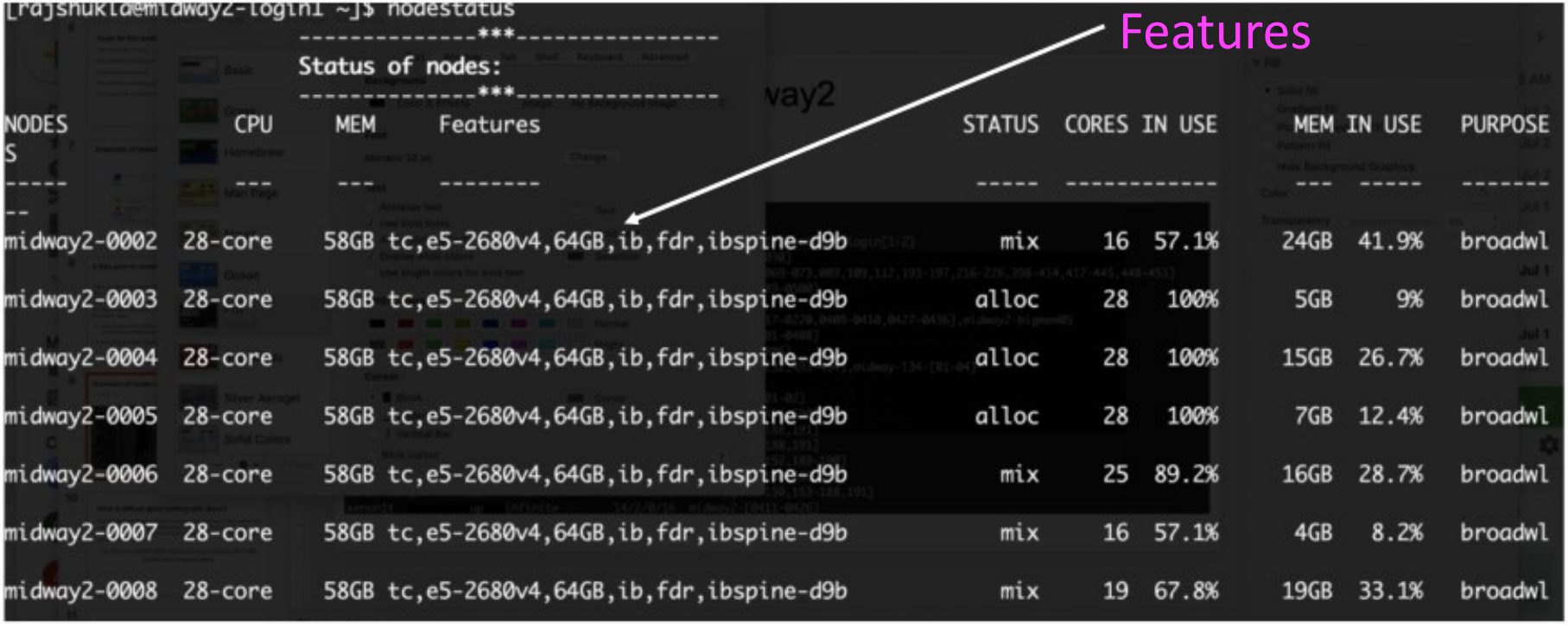
#SBATCH -exclusive

Add this in your batch script

#SBATCH -constraint=v100

How do I know the features of the node to use with #SBATCH -constraint?

nodestatus




```
[rajshukla@midway2-login1 ~]$ nodestatus
                ----------***----------
                Status of nodes:
                ----------***----------
NODES                CPU     MEM     Features                                  STATUS  CORES IN USE     MEM IN USE   PURPOSE
S
-----                ---     ---     --------                                  -----  ------------     ---  -----   -------
--
midway2-0002  28-core    58GB tc,e5-2680v4,64GB,ib,fdr,ibspine-d9b          mix     16  57.1%     24GB  41.9%   broadwl
midway2-0003  28-core    58GB tc,e5-2680v4,64GB,ib,fdr,ibspine-d9b          alloc   28  100%      5GB   9%      broadwl
midway2-0004  28-core    58GB tc,e5-2680v4,64GB,ib,fdr,ibspine-d9b          alloc   28  100%      15GB  26.7%   broadwl
midway2-0005  28-core    58GB tc,e5-2680v4,64GB,ib,fdr,ibspine-d9b          alloc   28  100%      7GB   12.4%   broadwl
midway2-0006  28-core    58GB tc,e5-2680v4,64GB,ib,fdr,ibspine-d9b          mix     25  89.2%     16GB  28.7%   broadwl
midway2-0007  28-core    58GB tc,e5-2680v4,64GB,ib,fdr,ibspine-d9b          mix     16  57.1%     4GB   8.2%    broadwl
midway2-0008  28-core    58GB tc,e5-2680v4,64GB,ib,fdr,ibspine-d9b          mix     19  67.8%     19GB  33.1%   broadwl
```

What resources should I ask for?

This depends on the code you are running

GPUs If you code is built to use gpus you can submit to the gpu partition. To request 1 gpu:

#SBATCH -p gpu2 --gres=gpu:1

What resources did my job actually use?

It is good practice to occasionally check what resources your job is using. For example if you are going to be submitting hundreds of similar jobs, you may save yourself a lot of waiting time in the queue by checking that you are not over requesting resources. Midway2 has a script to display the resources a job used:

```
jobinfo -j 99999
```

Replace 99999 with the job ID of the job you are interested in.

Why did my job fail?

- Ran out of Memory
- Ran out of Time
- Problem in your submit script
- Problem with your code
- Node failure

Why did my job fail?

- Ran out of Memory
- Ran out of Time
- Problem in your submit script
- Problem with your code
- Node failure

You can fix these

Why did my job fail?

- Ran out of Memory
- Ran out of Time
- Problem in your submit script
- Problem with your code

You can fix these

- Node failure

=> For node failure - please email help@rcc.uchicago.edu, if this happens

Job Priority

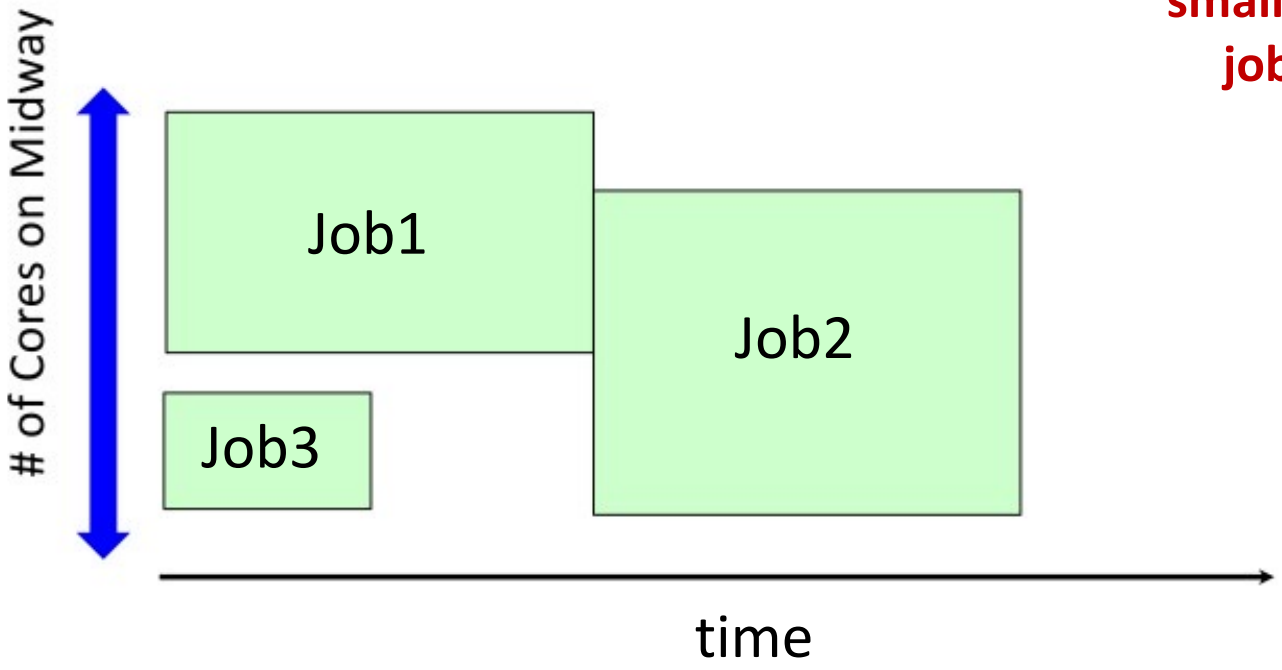
- Priority is calculates using a **Fairshare** algorithm
- Fairshare is function of
  - Requested wall clock, memory, nodes/cores, etc.
  - Length of time in queue
  - Number of jobs in a time window and per PI group
  - Backfill
  - Etc.

Job Priority

Backfill

Why has someone else's job started before mine?

The scheduler can fit smaller jobs in before big jobs are due to start

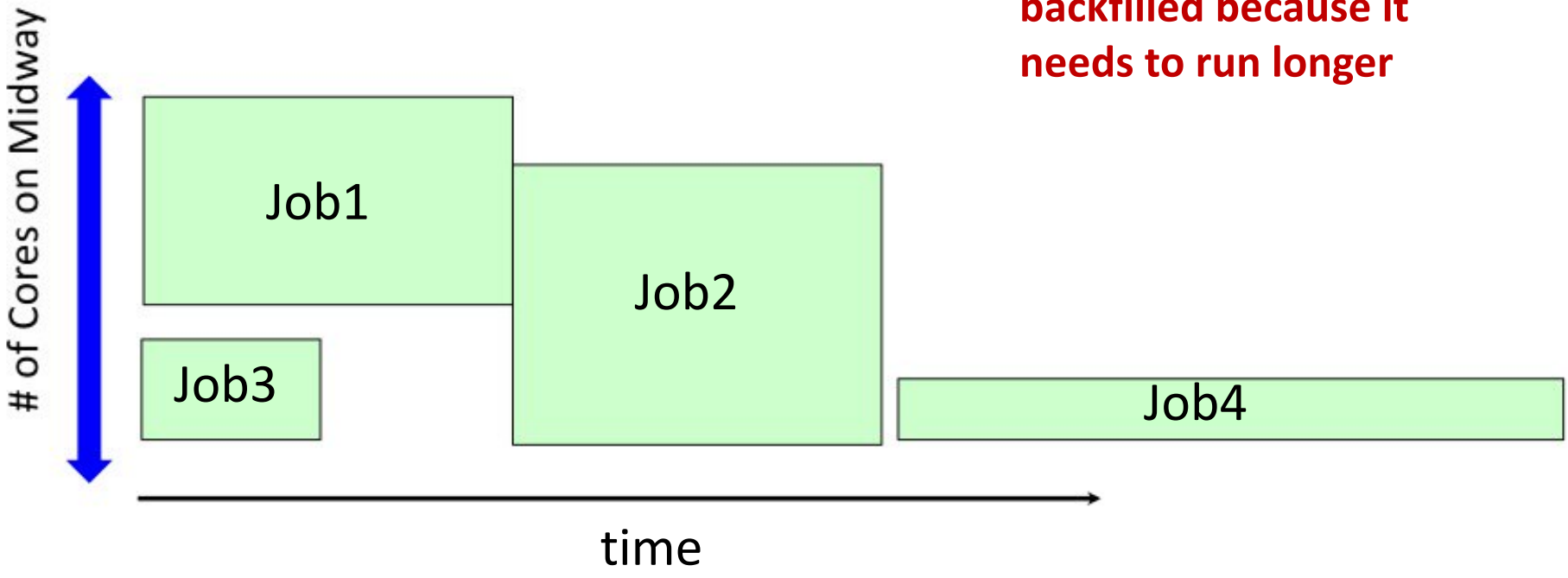


Job Priority

Backfill

Why has someone else's job started before mine?

Job4 while requesting less nodes, it can not be backfilled because it needs to run longer

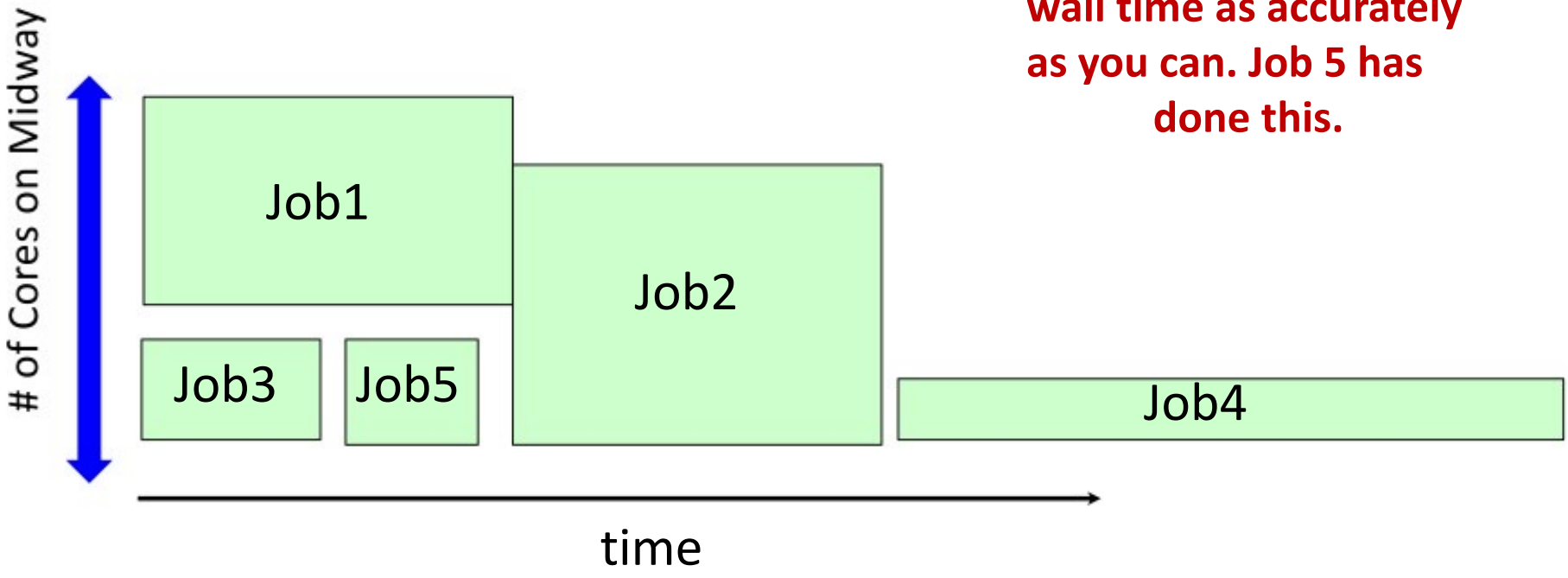


Job Priority

Backfill

Why has someone else's job started before mine?

You can take advantage of backfill by specifying wall time as accurately as you can. Job 5 has done this.

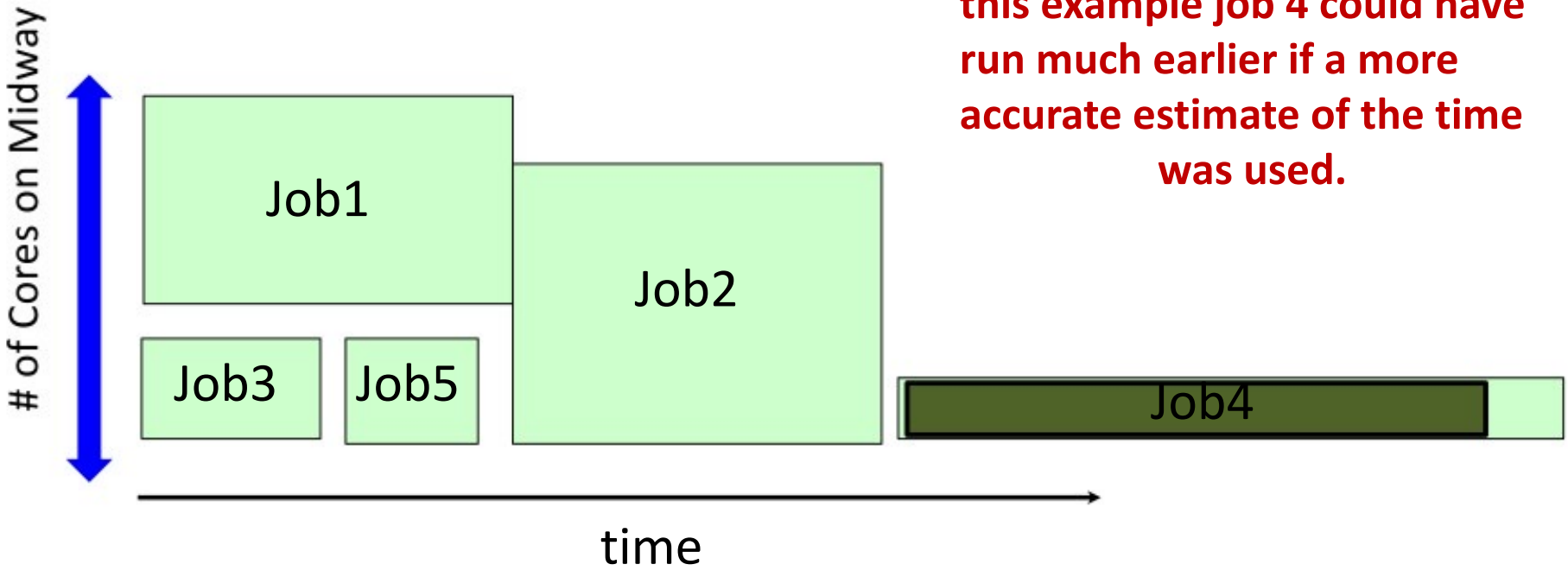


Job Priority

Backfill

Why has someone else's job started before mine?

You can take advantage of backfill by specifying wall time as accurately as you can. In this example job 4 could have run much earlier if a more accurate estimate of the time was used.



Job submission and monitoring

SLURM Commands

Command	Description
`sbatch script.sbatch`	Submits `script.sbatch` job script
`squeue -u $USER or myq`	Reports the status of your jobs
`sacct -u $USER`	Displays accounting data for your job(s)
`scancel jobid`	Cancels a running job or removes it from the queue
`scontrol show job jobid or jobinfo`	Displays details of a running job

Recommended online resources

- User guide on running jobs on Midway
  - https://rcc.uchicago.edu/docs/running-jobs/index.html
- Details Slurm documentation
  - https://slurm.schedmd.com/sbatch.html
- SLURM Cheat Sheet
  - https://slurm.schedmd.com/pdfs/summary.pdf

RCC Help

Contact:

- By email: help@rcc.uchicago.edu
- Web: rcc.uchicago.edu
- Phone: 773-795-2667
- In person:
  - 5607 S Drexel Avenue
  - Regenstein - Room 216
- Workshops and Tutorials:
- http://rcc.uchicago.edu/services/training.html

Thank You!